```
#!/usr/bin/env node
/**
 * FocusGhost — Trello Board Generator
 *
 * Creates the full FocusGhost project board via the Trello REST API.
 *
 * SETUP (one-time):
 *   1. Get your API key:  https://trello.com/power-ups/admin  → "New" → copy key
 *   2. Get your token:    https://trello.com/1/authorize?expiration=never&scope=
 *   3. Paste both below (or pass as env vars: TRELLO_KEY and TRELLO_TOKEN)
 *
 * RUN:
 *   node create-focusghost-trello.js
 *
 * Requires Node 18+ (uses native fetch).
 */

const API_KEY   = process.env.TRELLO_KEY   || "YOUR_API_KEY_HERE";
const API_TOKEN = process.env.TRELLO_TOKEN || "YOUR_TOKEN_HERE";

const BASE = "https://api.trello.com/1";
const auth = `key=${API_KEY}&token=${API_TOKEN}`;

// ─── helpers ──────────────────────────────────────────────────────────────

async function api(method, path, body = {}) {
  const url = `${BASE}${path}?${auth}`;
  const res = await fetch(url, {
    method,
    headers: { "Content-Type": "application/json" },
    body: method !== "GET" ? JSON.stringify(body) : undefined,
  });
  if (!res.ok) {
    const text = await res.text();
    throw new Error(`Trello ${method} ${path} → ${res.status}: ${text}`);
  }
  return res.json();
}

const post   = (path, body) => api("POST",   path, body);
const put    = (path, body) => api("PUT",    path, body);
const del    = (path)       => api("DELETE", path);
const get    = (path)       => api("GET",    path);
```

```
function sleep(ms) { return new Promise(r => setTimeout(r, ms)); }

// ── board data ──────────────────────────────────────────────────────────

// Labels: name → color
const LABELS_DEF = [
  { name: "Core Engine",  color: "green"  },
  { name: "UI / Screens", color: "blue"   },
  { name: "Ghost AI",     color: "purple" },
  { name: "Stuck Mode",   color: "orange" },
  { name: "Settings",     color: "yellow" },
  { name: "IPC / Arch",   color: "sky"    },
  { name: "Stretch",      color: "red"    },
  { name: "Docs / Team",  color: "pink"   },
];

// Lists in order
const LISTS_DEF = [
  "🗃️ Backlog",
  "🔨 Sprint 1 — Core Engine",
  "🎨 Sprint 2 — UI & Chat",
  "🧠 Sprint 3 — Ghost AI",
  "⚙️  Sprint 4 — Polish & Stretch",
  "✅ Done",
];

// Cards: [ listName, cardName, labels[], description, checklist[] ]
const CARDS_DEF = [

  // ── Sprint 1 — Core Engine ──────────────────────────────────────────

  [
    "🔨 Sprint 1 — Core Engine",
    "Project scaffold (Electron Forge + Vite + React + TS + Tailwind)",
    ["Core Engine"],
    "Bootstrap the project with Electron Forge using the Vite + React + TypeScrip
    [
      "Run `npm init electron-app@latest focusghost -- --template=vite-typescript
      "Install tailwindcss, postcss, autoprefixer",
      "Configure tailwind.config.js and globals.css",
      "Confirm Electron window opens with React renderer",
      "Add ESLint + Prettier config",
      "Push initial commit to repo",
    ],
  ],

  [
```

```
  "🔨 Sprint 1 — Core Engine",
  "IPC contract definition (Main ↔ Renderer)",
  ["IPC / Arch"],
  "Define and document all IPC channel names, payload shapes, and directionalit
  [
    "Create `/src/shared/ipc.ts` with channel name constants",
    "Define TypeScript interfaces for every payload",
    "Document: session:start, session:end, window:switch, nudge:trigger, stuck:
    "Set up contextBridge preload exposing safe API to renderer",
    "Share doc with team",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "active-window integration — OS window polling loop",
  ["Core Engine"],
  "Install `active-win` and set up a polling loop in the main process (every 1-
  [
    "Install active-win (`npm install active-win`)",
    "Create `/src/main/windowTracker.ts`",
    "Poll every 1500ms, compare to previous active window",
    "On change: emit `window:switch` IPC event with { app, title, timestamp }",
    "Handle macOS Accessibility permission request",
    "Test on Windows and macOS",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "Session state machine (Idle → Active → Drifting → Recap)",
  ["Core Engine", "IPC / Arch"],
  "Implement the core session FSM. States: IDLE, ACTIVE, DRIFTING, INACTIVE, RE
  [
    "Define SessionState enum: IDLE | ACTIVE | DRIFTING | INACTIVE | RECAP",
    "Create `/src/main/sessionMachine.ts`",
    "Implement state transitions with guards",
    "Expose current state via IPC to renderer",
    "Wire session:start and session:end IPC handlers",
    "Unit test state transitions",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "Per-app time tracker & switch counter",
  ["Core Engine"],
```

```
  "For each session, maintain a map of appName → { totalMs, switchCount }. Upda
  [
    "Create AppTimeMap type: Record<string, { totalMs: number; switches: number
    "Update map on every window:switch event",
    "Track current app start time to compute elapsed on next switch",
    "Expose live stats snapshot via IPC (stats:update)",
    "Persist final map to store on session end",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "Inactivity detection timer",
  ["Core Engine"],
  "Track time since last window switch or user input. If no activity for N seco
  [
    "Add inactivity timer alongside polling loop",
    "Reset timer on every window:switch event",
    "Configurable threshold from settings store",
    "On timeout: emit `session:inactive` IPC event",
    "Display inactive time separately from drift time in UI",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "Local data persistence (electron-store / SQLite)",
  ["Core Engine"],
  "Store session records, app time maps, nudge history, and settings locally. E
  [
    "Install electron-store",
    "Define store schema: sessions[], settings, appCategories",
    "Write saveSession() and loadSessions() helpers",
    "Write getSettings() and updateSettings() helpers",
    "Test persistence across app restarts",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "App categorization system (Focus / Research / Distraction)",
  ["Core Engine"],
  "Maintain a map of app name patterns → category. Used for drift risk scoring
  [
    "Define AppCategory enum: FOCUS | RESEARCH | DISTRACTION | UNKNOWN",
    "Create default category map (VS Code, terminal → FOCUS; Chrome/Safari → RE
    "Expose category lookup function used by drift scorer",
```

```
    "Store overrides in electron-store",
  ],
],

[
  "🔨 Sprint 1 — Core Engine",
  "Drift risk scorer",
  ["Core Engine"],
  "Compute a Low / Medium / High drift risk score in real time. Inputs: switch
  [
    "Define DriftRisk enum: LOW | MEDIUM | HIGH",
    "Score algorithm: weight recent switches + distraction % + inactivity",
    "Recalculate every 30 seconds",
    "Emit score via IPC for renderer to display",
    "Determine nudge trigger threshold per sensitivity preset",
  ],
],

// ── Sprint 2 — UI & Chat ──────────────────────────────────────────

[
  "🎨 Sprint 2 — UI & Chat",
  "Screen 01 — Task Declaration",
  ["UI / Screens"],
  "Pre-session setup screen. User types a task name, selects duration (15/30/45
  [
    "Task name text input with placeholder",
    "Duration selector: 4 pill buttons (15m selected by default)",
    "Ghost mascot centered above input",
    "'start focus session' button (disabled until task name is non-empty)",
    "↵ to start hint text",
    "Wire to session:start IPC on confirm",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Screen 02 — Active Session panel",
  ["UI / Screens"],
  "Live session view showing task name, countdown timer, current app (with FOCU
  [
    "Task name + countdown timer header",
    "Current app name + FOCUS/DRIFT badge",
    "Stats row: Switches, Focus Time, Drift Time (live)",
    "Drift Risk indicator: LOW / MEDIUM / HIGH with color",
    "Recent Activity list: app name, time in app, switch count",
    "'simulate switch' + 'end session' dev buttons",
```

```
    "Subscribe to stats:update IPC",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Screen 03 — Ghost Chat panel",
  ["UI / Screens"],
  "Full-session chat log. Shows ghost nudge messages, pattern notices, sprint o
  [
    "Scrollable message list (ghost bubbles + timestamps)",
    "Action message variant: nudge with Accept / Dismiss CTA buttons",
    "Pattern notice variant: amber label + insight text",
    "Stuck Mode variant: red border card with input + submit",
    "Freeform chat input at bottom with send button",
    "Auto-scroll to latest message",
    "Subscribe to nudge:trigger and stuck:activate IPC",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Screen 04 — Session Recap",
  ["UI / Screens"],
  "Post-session summary. Shows focus time, drift time, switches, nudge count, t
  [
    "SESSION COMPLETE header + task name + total time",
    "4-stat grid: Focus Time, Drift Time, Switches, Nudges",
    "Top Apps ranked list with colored time bars",
    "Ghost Insight card (AI sentence, fetched on session end)",
    "'start new session' teal CTA button",
    "Wire 'start new session' to reset state → Screen 01",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Collapsed always-on-top bar",
  ["UI / Screens"],
  "Single-line horizontal strip when window is minimized. Shows: ghost icon, ta
  [
    "Compact bar component at ~36px height",
    "Ghost icon + task name (truncated) + timer",
    "Current app name with dot indicator (green = focus, red = drift)",
    "Opacity toggle icon",
    "Expand icon → restores full window",
    "Drifting state: ghost icon turns pink/red",
```

```
    "Wire always-on-top via Electron BrowserWindow flag",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Window opacity controls",
  ["UI / Screens"],
  "Allow the user to make the FocusGhost window semi-transparent so they can se
  [
    "Opacity slider in Settings (0–100%)",
    "Quick cycle button in collapsed bar (100% → 75% → 45%)",
    "Apply via Electron `win.setOpacity()`",
    "Persist last opacity to electron-store",
  ],
],

[
  "🎨 Sprint 2 — UI & Chat",
  "Ghost mascot component (static + animated states)",
  ["UI / Screens", "Stretch"],
  "Ghost character used across all screens. Static version required for MVP. An
  [
    "SVG/PNG ghost asset (teal, rounded, smiley face)",
    "Static ghost component used in Screen 01, chat bubbles, collapsed bar",
    "[Stretch] Lottie animation with states: idle, drift, stuck, success",
    "[Stretch] Trigger state transitions from session FSM events",
  ],
],

// ── Sprint 3 — Ghost AI ──────────────────────────────────────────

[
  "🧠 Sprint 3 — Ghost AI",
  "Gemini API integration + shared prompt runner",
  ["Ghost AI"],
  "Wire up the Gemini API in the main process. Build a shared `promptGemini(sys
  [
    "Install @google/generative-ai SDK",
    "Create `/src/main/gemini.ts` with promptGemini() helper",
    "System prompt includes: task name, session stats snapshot, app category co
    "Handle rate limit + timeout errors gracefully",
    "Keep API key in main process only — never expose to renderer",
    "Log prompts + responses to session store for debugging",
  ],
],
```

```
[
  "🧠 Sprint 3 — Ghost AI",
  "Drift nudge prompt templates",
  ["Ghost AI"],
  "Define prompt templates for each nudge trigger type: app-switch drift, inact
  [
    "Template: switch-drift nudge (references task + switch count)",
    "Template: inactivity nudge (gentle 'still there?')",
    "Template: check-in nudge (positive reinforcement for focus streak)",
    "Template: sprint offer (propose 12-min focus sprint)",
    "All templates inject: { taskName, switchCount, focusTime, currentApp }",
    "Test each template with sample data",
  ],
],

[
  "🧠 Sprint 3 — Ghost AI",
  "Pattern notice generation (mid-session insight)",
  ["Ghost AI"],
  "Periodically generate a behavioral pattern observation during the session. E
  [
    "Trigger every 15 minutes of active session",
    "Prompt template: analyze app timeline → surface one non-obvious pattern",
    "Inject full app time map into prompt",
    "Display in Ghost Chat with PATTERN NOTICED label",
    "Only fire if there's enough data (>10 min elapsed)",
  ],
],

[
  "🧠 Sprint 3 — Ghost AI",
  "Ghost Chat freeform Q&A",
  ["Ghost AI"],
  "Allow the user to type any message to the ghost and get a contextual respons
  [
    "Chat input in Screen 03 sends message via IPC",
    "Main process passes message + session context to Gemini",
    "Response returned via IPC → appended to chat log",
    "Loading indicator while waiting for response",
    "Cap conversation history at last 10 turns to manage token count",
  ],
],

[
  "🧠 Sprint 3 — Ghost AI",
  "Ghost Insight — end-of-session AI summary",
  ["Ghost AI"],
```

```
  "Generate a single, specific, human-sounding sentence summarizing the session
  [
    "Trigger on session:end",
    "Prompt template: given full session stats → write one punchy insight sente
    "Inject: focusTime, driftTime, switchCount, topApps, nudgeCount, task",
    "Display in Session Recap ghost card",
    "Show loading spinner in recap while insight generates",
  ],
],

// ── Sprint 3 — Stuck Mode ──────────────────────────────────────────

[
  "🧠 Sprint 3 — Ghost AI",
  "Stuck Mode trigger logic",
  ["Stuck Mode"],
  "Detect the 'debugging drift' pattern: rapid cycling between a small set of 3
  [
    "Track app cycle patterns: detect same 2-3 apps repeating",
    "Trigger if: ≥6 switches among ≤3 apps in last 4 minutes AND avg dwell < 90
    "Guard: only trigger once per 10-minute window",
    "Emit stuck:activate IPC event to renderer",
    "Configurable sensitivity tied to global nudge sensitivity setting",
  ],
],

[
  "🧠 Sprint 3 — Ghost AI",
  "Stuck Mode overlay UI + input form",
  ["Stuck Mode", "UI / Screens"],
  "Card overlay in Ghost Chat triggered by stuck:activate. Prompts user to desc
  [
    "Stuck Mode card: amber/orange border, warning icon",
    "Header: 'Noticed you've been on the same problem for a while.'",
    "Single text input: 'What are you having trouble with?'",
    "Placeholder: 'e.g., understanding recursion in this function'",
    "'Get Unstuck Suggestion' submit button",
    "Loading state while Gemini responds",
    "Render AI response inline in chat as a structured card",
  ],
],

[
  "🧠 Sprint 3 — Ghost AI",
  "Stuck Mode — Gemini unstuck prompt + structured response",
  ["Stuck Mode", "Ghost AI"],
  "Prompt template for Stuck Mode. Takes user's problem description + task cont
```

```
  [
    "Prompt template: { taskName, stuckDescription, currentApp, recentApps }",
    "Instruct model to respond with: 1 clarifying reframe + 2–3 concrete next s
    "Response displayed as structured card in Ghost Chat",
    "Follow-up: ghost asks 'Did that help?' with Yes / Still stuck options",
  ],
],

// ── Sprint 4 — Polish & Stretch ──────────────────────────────────────────

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "Settings panel — full implementation",
  ["Settings"],
  "Complete Settings screen covering all configurable options: appearance, nudg
  [
    "Back arrow → returns to previous screen",
    "APPEARANCE section: opacity slider, accent color (teal/violet/amber), alwa
    "NUDGES section: sensitivity preset (gentle/balanced/strict), drift thresho
    "SESSION DEFAULTS: default duration, ghost personality (encouraging / neutr
    "All settings persist via electron-store",
    "Live preview: accent color changes apply immediately",
  ],
],

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "Nudge sensitivity presets (Gentle / Balanced / Strict)",
  ["Settings", "Core Engine"],
  "Map three named presets to specific parameter bundles. Gentle: high drift th
  [
    "Define preset bundles: { driftThresholdMin, switchTriggerCount, inactivity
    "Gentle: 5 min / 8 switches / 90s",
    "Balanced: 3 min / 5 switches / 60s",
    "Strict: 1.5 min / 3 switches / 30s",
    "Apply preset on selection, allow manual override via sliders",
  ],
],

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "Accent theme system (teal / violet / amber)",
  ["Settings", "UI / Screens"],
  "Three accent color themes applied globally via CSS custom properties. Active
  [
    "Define CSS vars: --accent, --accent-dim, --accent-text",
    "Teal: #00D4D4, Violet: #A855F7, Amber: #F59E0B",
```

```
    "Apply theme class to document root on settings change",
    "Persist to store, restore on app launch",
  ],
],

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "Demo script + seed data mode",
  ["Docs / Team"],
  "Script for live demo at hackathon / presentation. A dev mode that simulates
  [
    "Add `--demo` flag to Electron launch",
    "Simulated switch sequence: VS Code → Twitter → YouTube → VS Code → Chrome"
    "Auto-trigger nudge at 2-min mark",
    "Auto-trigger Stuck Mode at 4-min mark",
    "Auto-end session at 6-min mark with sample recap data",
    "Write full spoken demo script in team doc",
  ],
],

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "[Stretch] ElevenLabs voice nudges",
  ["Stretch"],
  "Ghost speaks nudge messages aloud using ElevenLabs TTS. Adds presence — espe
  [
    "Install ElevenLabs Node SDK",
    "Select ghost voice (soft, slightly playful)",
    "TTS on: drift nudges, sprint offers, Stuck Mode trigger",
    "Volume control + mute toggle in Settings",
    "Graceful fallback if API call fails",
  ],
],

[
  "⚙️  Sprint 4 — Polish & Stretch",
  "[Stretch] Lottie / CSS ghost mascot animations",
  ["Stretch"],
  "Animated ghost reacting to session state. Idle: slow float. Drifting: jitter
  [
    "Source or design ghost Lottie animation (4 states)",
    "Integrate `lottie-react` or CSS keyframe fallback",
    "Map session FSM states to animation states",
    "Ensure animation doesn't distract — subtle by default",
  ],
],
```

```
// ── Backlog ──────────────────────────────────────────────────────────

[
  "🧊 Backlog",
  "Multi-session history view",
  ["UI / Screens"],
  "Browse past sessions with stats and ghost insights. Could be a simple list v
  [],
],

[
  "🧊 Backlog",
  "Focus streak tracking (cross-session)",
  ["Core Engine"],
  "Track consecutive days with a session above a focus threshold. Display strea
  [],
],

[
  "🧊 Backlog",
  "Custom app category editor UI",
  ["Settings", "UI / Screens"],
  "Let users reclassify apps (e.g., mark Notion as FOCUS, mark GitHub as RESEAR
  [],
],

[
  "🧊 Backlog",
  "Windows + Linux testing pass",
  ["Core Engine"],
  "active-win behaves differently per OS. Full regression test on Windows and L
  [],
],

// ── Docs ─────────────────────────────────────────────────────────────

[
  "🧊 Backlog",
  "Team doc: IPC message contract",
  ["Docs / Team", "IPC / Arch"],
  "Living document listing every IPC channel, payload shape, direction, and whi
  [
    "Table: channel | direction | payload type | owner | notes",
    "Channels: session:start, session:end, window:switch, stats:update, nudge:t
    "Host in Notion / GitHub Wiki / Google Doc — link in README",
  ],
],
```

```
    [
      "🥶 Backlog",
      "Team doc: Gemini prompt templates",
      ["Docs / Team", "Ghost AI"],
      "Centralized doc of all prompt templates with variable placeholders, expected
      [
        "drift-nudge template",
        "pattern-notice template",
        "stuck-mode template",
        "ghost-insight (recap) template",
        "freeform-chat system prompt",
        "Include sample outputs for each",
      ],
    ],

    [
      "🥶 Backlog",
      "Team doc: component list with owner names",
      ["Docs / Team"],
      "Spreadsheet or table mapping every React component and main-process module t
      [],
    ],
];

// ─── main ──────────────────────────────────────────────────────────────────

async function main() {
  console.log("🚀  Creating FocusGhost Trello board...\n");

  // 1. Create board
  const board = await post("/boards", {
    name: "FocusGhost 👻",
    desc: "Desktop AI focus companion — Electron + React + Gemini",
    defaultLists: false,
    prefs_background: "blue",
  });
  console.log(`✅  Board created: ${board.url}`);

  // 2. Delete default lists if any sneak through
  const existingLists = await get(`/boards/${board.id}/lists`);
  for (const list of existingLists) {
    await put(`/lists/${list.id}/closed`, { value: true });
  }

  // 3. Create labels
  console.log("🏷️   Creating labels...");
```

```
const labelMap = {};
for (const { name, color } of LABELS_DEF) {
  await sleep(120);
  const label = await post("/labels", { name, color, idBoard: board.id });
  labelMap[name] = label.id;
  process.stdout.write(`   + ${name}\n`);
}

// 4. Create lists (in reverse so Trello order is correct)
console.log("\n📋  Creating lists...");
const listMap = {};
for (const listName of [...LISTS_DEF].reverse()) {
  await sleep(150);
  const list = await post("/lists", {
    name: listName,
    idBoard: board.id,
    pos: "top",
  });
  listMap[listName] = list.id;
  process.stdout.write(`   + ${listName}\n`);
}

// 5. Create cards + checklists
console.log("\n🏴‍☠️  Creating cards...");
for (const [listName, cardName, labels, desc, checklist] of CARDS_DEF) {
  await sleep(200);
  const idLabels = labels.map((l) => labelMap[l]).filter(Boolean);
  const card = await post("/cards", {
    name: cardName,
    desc,
    idList: listMap[listName],
    idLabels,
  });
  process.stdout.write(`   + [${listName}] ${cardName}\n`);

  if (checklist && checklist.length > 0) {
    await sleep(150);
    const cl = await post("/checklists", {
      idCard: card.id,
      name: "Tasks",
    });
    for (const item of checklist) {
      await sleep(100);
      await post(`/checklists/${cl.id}/checkItems`, { name: item });
    }
  }
}
```

```
  console.log(`\n🎉  Done! Open your board:\n    ${board.url}\n`);
}

main().catch((err) => {
  console.error("❌  Error:", err.message);
  process.exit(1);
});
```